Análisis de distintas implementaciones para tablas Hash

Marcelo Fort Muñoz

Algoritmos y Estructuras de Datos
marcelo.fort@rai.usc.es

I. INTRODUCCIÓN

Las tablas hash son unas estructuras de datos ampliamente utilizadas para el almacenamiento de datos que permiten almacenar y recuperar información de forma eficiente a partir de unas claves generadas por una función hash.

Sin embargo, no todas las tablas hash son iguales, y su eficacia depende de varios factores, como el tipo de estrategia de resolución de colisiones, el tamaño de la tabla y el número de elementos almacenados. En este artículo se presenta un estudio comparativo con el ejemplo de la tabla pacientes_vacunodromo.txt de diferentes estrategias de resolución de colisiones (recolocación y encadenamiento), tamaños de tabla (N), factor de ponderación (K) y, en el caso de la recolocación, si esta se realiza de forma lineal o cuadrática (a).

El objetivo es determinar la mejor estrategia de implementación de implementación, analizar el comportamiento de las tablas hash y estudiar este en función de la eficiencia y recursos.

II. CONCEPTOS BÁSICOS

Antes de entrar a analizar el comportamiento de las tablas, conviene definir un par de términos que serán utilizados en el resto del documento:

- **Colisión**: Dos o más claves son asignadas al mismo **hash** (posición en la tabla). La colisión reduce la eficacia de la tabla hash, ya que conlleva un aumento de pasos y, consecuentemente, del tiempo requerido para operar. En el caso de la tabla de encadenamiento, se insertará el elemento al final de la lista. Mientras que en el caso de la recolocación se intentará buscar un hueco para dicha información. En el código se exploran las estrategias de recolocación lineal y cuadrática.
- **Hash**: Cadena de texto codificada que representa a una serie de datos de forma única e irrepetible. Este es obtenido mediante una función matemática que transforma los datos de entrada en una salida de longitud fija o variable, dependiendo del tipo de función.

Las abreviaturas utilizadas son:

- **impl**: implementación

III. DETALLES ESPECÍFICOS DE LA IMPLEMENTACIÓN

Es relevante antes de entrar al análisis de la información obtenida tras el análisis comentar los detalles de los **cambios que ha sufrido el código** de las tablas para poder llevar el conteo de tanto las colisiones como de las operaciones extra.
Los cambios realizados en el código fuente son los que siguen:

1. Tanto para el caso de la impl. con la estrategia de recolocación como en el de la estrategia de encadenamiento, se ha modificado el main de forma que se pueda ejecutar el código desde la consola sin la necesidad de ejecutar este en modo interactivo y se han modificado las impresiones a consola para que la salida sea fácilmente exportada a un programa de gestión de datos.
2. Se ha alterado la firma de las funciones de ambas impl. para permitir que se cambie el campo usado como hash fácilmente sin necesitar alterar el código fuente en cada ejecución. Para esto se ha creado una función privada llamada `_SeleccionarClave`. Esta escoge en base a la entrada de un carácter un campo de la estructura pasada como parámetro y lo devuelve.
3. Se ha modificado la firma de todas las funciones que necesitaban usar la clave para poder pasar que campo se ha de usar como clave.
4. Se ha implementado un **`makefile`** para poder serializar la realización de experimentos y la recolección de los resultados de estos.

5. En la implementación de **recolocación**, en el caso de **`_posicionBuscar`**, se ha pasado a aceptar el parámetro `nPasosExtraB` por referencia. Esta es aumentada cuando termine o encuentre la clave.
6. También en la implementación de recolocación, en el caso de **`_PosicionInsertar`** se han añadido por referencia los parámetros `hayColisionI` y `nPasosExtraI`. El primero es actualizado cuando hay una colisión y el segundo en caso de que falle al encontrar la clave o se encuentre con una clave vacía o borrada.
7. En el caso de **`insertarHash`**, en ambas implementaciones se ha modificado la firma para devolver un entero y, se ha añadido una variable bandera que es devuelta al final de la función con un valor positivo en el caso de que hubiera acaecido una colisión y 0 en caso contrario. En el caso específico de recolocación, se ha añadido por parámetro la variable `nPasosExtraI`, que es pasada a su vez a **`_PosicionInsertar`**.
8. Para la función **`buscarHash`**, se ha modificado la firma para aceptar como parámetro `nPasosExtraB`. Esta variable es incrementada cada vez que la función falla en encontrar la clave. Este cambio es así para encadenamiento. En el caso de recolocación, esta es simplemente pasada a `_posicionBuscar`.
9. Para la función **`esMiembroHash`** se han llevado a cabo los mismos cambios que en `buscarHash`.
10. Para la función **`borrarHash`**, se ha modificado la firma para aceptar como parámetro `nPasosExtraB`. Esta variable es incrementada con cada eliminación. Este cambio es así para encadenamiento. En el caso de recolocación, esta es simplemente pasada a `_posicionBuscar`.
11. Ambas cabeceras han sido modificadas para soportar estos cambios y se ha creado un `enum` para los tipos de claves aceptados: nombre, nick y correo.
12. En los main de las dos versiones se han creado variables para llevar contabilidad de los distintos tipos de colisiones y pasos adicionales, se han modificado las firmas de las funciones `insercionArchivo` y `busquedaArchivo` para aceptar las variables e, incrementado el valor de los contadores previamente mencionados cuando ha sido necesario.

IV. EXPERIMENTOS REALIZADOS Y RESULTADOS

1. Influencia del tamaño (N) elegido para la tabla hash en el proceso de inserción

En este experimento se desea observar el comportamiento de la función **hash tipo 2** en las implementaciones de encadenamiento y recolocación. En esta segunda se comprobará el caso lineal con a = 1.

Como primer paso, se han tenido que seleccionar seis valores para N. Debemos recordar que los valores más adecuados para N son aquellos que tienen un factor de carga (L) menor o igual a 0.75 (para la impl. de encadenamiento) o menor o igual a 0.5 (para la impl. de recolocación). Además, también sabemos que los números primos tendrán mucho mejor rendimiento que los no primos.

En una primera selección hemos escogido números primos con factores de carga cercanos a 0.2 0.3 … hasta 0.8 y también algunos no primos. Como teníamos que escoger finalmente seis, hemos elegido varios primos con factores de carga por debajo de los límites recomendados y adicionalmente un primo con el factor de carga superior al recomendado y un no primo. Los valores seleccionados finalmente se pueden ver en la siguiente tabla:

TABLA 0
CARACTERÍSTICAS DE LOS NÚMEROS SELECCIONADOS PARA N

N	Es primo	L	Observaciones
12503	Sí	0.800	L muy alto para ambos tipos
16673	Sí	0.600	L muy alto para recolocación
25013	Sí	0.400	
33343	Sí	0.300	
49999	Sí	0.200	
44444	No	0.225	L es similar al 49999 pero este no es primo

Aunque en el enunciado se nos indica que podemos escoger distintos valores para N en el caso de recolocación y encadenamiento, hemos decidido usar los mismos valores.

Los resultados obtenidos para los diferentes casos se pueden observar en esta tabla:

TABLA 1
INFLUENCIA DE N EN EL PROCESO DE INSERCIÓN USANDO LA FUNCIÓN HASH2

N	12503	16673	25013	33343	49999	44444
Encadenamiento						
nColisionesI	3136	2379	1792	1359	**977**	3430
Recolocación Simple (a=1)						
nColisionesI	4024	2884	2061	1495	**1046**	3434
nPasosExtraI	21576	6824	3488	2029	**1344**	4606

Se puede concluir que el resultado dado por N=49.999 es el que mejor comportamiento ha presentado, tanto para encadenamiento como para recolocación simple, dado que este ha arrojado el menor número tanto de Colisiones como de pasos extra en ambas implementaciones. **Estos "mejores resultados" se han marcado en negrita en la tabla**. Dicho valor de N (49.999, con L=0.225), nos permite obtener hasta 2-3 veces mejores resultados que los que se obtienen con otros primos con valores de L que aún mayores, seguirían dentro de los rangos recomendado (0.75 para encadenamiento, 0.5 para recolocación).

Incluso llegamos a encontrar valores aún peores para otros primos con L fuera de los rangos recomendados, y para el 44444 (no primo). Se han marcado todos estos valores en un color gris más claro, para identificar que NO siguen la recomendación sobre los valores de N a seleccionar. Como número totalmente fuera de rango, cabe destacar nos más de 21000 pasos extra que son necesarios para el método de recolocación simple con N=12503 (L=0.8, claramente por encima del límite recomendado de 0.5). Se han marcado finalmente los peores resultados con un subrayado.

Nota: Estas mismas marcas se realizarán en el resto de las tablas:

- **Mejores resultados**
- Peores resultados
- Resultados obtenidos para valores de N ya de antemano "no recomendables" por superar el límite de L o no ser primos.

2. *Influencia de la función hash (1-2-3) y de la clave en el proceso de inserción (nColisionesI)*

Para analizar la influencia de la función hash en el número de colisiones en el proceso de inserción vamos a detallar como funciona cada una de estas funciones hash:

1. **Función Hash 1:** Esta calcula el hash sumando los códigos ASCII de cada uno de los caracteres de la cadena pasada por referencia. Finalmente calcula el modulo respecto a N para que no haya un hash mayor al número de entradas en la tabla. En este caso, una palabra del estilo ABCD tendría el mismo hash que DCBA o BDCA, ya que la suma de los códigos ASCII de las letras serán los mismos en todos los casos y, por lo tanto, su suma invariable. Incluso aún peor que esto, resulta que, una palabra como AADD tendría exactamente el mismo hash que cualquiera de las especificadas anteriormente, puesto que la suma de sus códigos ASCII es igual a la de los mencionados anteriormente.

2. **Funciones Hash 2:** Estas funciones calculan el hash haciendo la suma ponderada (similar al sistema de posicionamiento de los números arábigos) de los valores ASCII de los caracteres de la cadena pasada por referencia. Al ser ponderada la suma, el orden de entrada de los valores ASCII va a condicionar el resultado final, por lo que cadenas con mismos caracteres en distinto orden tendrán hashes diferentes. Como factor de ponderación(K), en este caso se utiliza K = 256, que es el número de posibles distintos valores que puede tomar un dato tipo char.

3. **Función Hash 3:** El funcionamiento de la función Hash 3 es idéntica a la de la función Hash 2, pero con la excepción de que en este caso se pasa un valor K arbitrario.

Se nos ha solicitado que analicemos las 3 funciones hash, y, para la función Hash 3 en particular, se nos ha pedido que escojamos varios valores de K. Para nuestro caso particular, **hemos escogido 500, 503, 1024 y 1031 como valores adicionales de K**.

El número de colisiones durante el proceso de inserción en encadenamiento para cada caso se muestra en la siguiente tabla:

TABLA 2
INFLUENCIA DE LA FUNCIÓN HASH EN EL PROCESO DE INSERCIÓN EN ENCADENAMIENTO – CLAVE=NICKNAME

Func. Hash	K	N	12503	16673	25013	33343	49999	44444
1	-		9271	9271	9271	9271	9271	9271
2	256		3136	2379	1792	1359	**977**	3430
3	500		3080	2484	1769	1398	1004	3380
3	503		3163	2403	1805	1315	**879**	1031
3	1024		3092	2487	1769	1283	1024	3348
3	1031		3045	2396	1823	1370	**969**	1050

Vemos que las funciones Hash 2 y 3 son las que mejor se comportan en general, pero que, la selección de diferentes valores de K no ha afectado demasiado a los resultados finales, con lo que entendemos que los valores de K no afectan especialmente en el proceso (al menos quedándonos como mínimo en el 256).

Por su parte, la función Hash 1 tiene un comportamiento mucho peor, apareciendo entre 3 y 10 veces más colisiones que con las otras dos funciones Hash. Además, vemos que el número de colisiones NO ha cambiado en función del valor de K (cosa que sí sucedía en las otras dos funciones de Hash).

La función Hash 1 es especialmente mala para este problema en concreto debido a la pequeña variabilidad que existe entre los distintos valores que se usan para como cadena de entrada (los nicks de los usuarios). Todos tienen la forma de **una arroba, unas 8 a 12 letras minúsculas y en ocasiones un número final**. Esto lleva a que, como ya contábamos más arriba en la explicación de cómo se calculan los hashes en este método (dando con ello **una baja dispersión en los resultados**), haya una **alta probabilidad de que aparezcan colisiones.**

TABLA 3
INFLUENCIA DE LA FUNCIÓN HASH EN EL PROCESO DE INSERCIÓN EN RECOLOCACIÓN LINEAL (A=1) – CLAVE=NICKNAME

Func. Hash	K	Valores	N	12503	16673	25013	33343	49999	44444
1	-	nColisionesI		9784	9784	9784	9784	9784	9784
1	-	nPasosExtraI		46720557	46720557	46720557	46720557	46720557	46720557
2	256	nColisionesI		4024	2884	2061	1495	**1046**	3434
2	256	nPasosExtraI		21576	6824	3488	2029	**1344**	4606
3	500	nColisionesI		3968	3005	2029	1520	1097	3384
3	500	nPasosExtraI		21122	6729	3511	2136	1449	4466
3	503	nColisionesI		4089	2905	2056	1448	**922**	1111
3	503	nPasosExtraI		20387	6787	3507	1967	**1168**	1436
3	1024	nColisionesI		3984	2979	2027	1400	1091	3353
3	1024	nPasosExtraI		19245	7232	3467	2026	1403	4340
3	1031	nColisionesI		3902	2899	2064	1502	**1037**	1131
3	1031	nPasosExtraI		18093	7008	3326	2166	**1302**	1474

En el caso del proceso de inserción en recolocación lineal, se aprecian resultados muy parecidos a los de la inserción en encadenamiento. Ganan de nuevo las funciones Hash 2 y 3, vemos poca diferencia entre unos valores y otros de K, y por supuesto valores de N primos con factores de carga bajos son mejores que el resto.

Queremos destacar aquí los resultados obtenidos por la función **Hash 1** en el recuento de **pasos extra para la inserción**, que se dispara hasta los más de **460 millones**, en comparación a los pocos miles que obtienen los métodos de Hash 2 y 3.

Si comparamos los resultados obtenidos en el proceso de inserción en recolocación, comparado con el proceso de inserción en encadenamiento, vemos que el de encadenamiento (analizado en la tabla 2) ha presentado resultados en general ligeramente mejores que el recién analizado.

Comparación del campo usado como clave para la función hash

Procederemos ahora a analizar la influencia del campo clave seleccionado. Para ello, usaremos el correo electrónico en lugar de el alias del usuario. Los resultados obtenidos para ambos procesos de inserción se pueden visualizar en las siguientes dos tablas:

TABLA 4
INFLUENCIA DE LA FUNCIÓN HASH EN EL PROCESO DE INSERCIÓN EN ENCADENAMIENTO – CLAVE=CORREO

Func. Hash	K	N	12503	16673	25013	33343	49999	44444
1	-		8787	8787	8787	8787	8787	8787
2	256		3073	2464	1770	1331	**882**	1145
3	500		3139	2517	1729	1433	**912**	1210
3	503		3084	2484	1764	1323	**885**	1029
3	1024		3182	2442	1756	1370	**908**	1176
3	1031		3105	2469	1793	1388	931	1083

TABLA 5
INFLUENCIA DE LA FUNCIÓN HASH EN EL PROCESO DE INSERCIÓN EN RECOLOCACIÓN LINEAL (A=1) – CLAVE=CORREO

Func. Hash	K	Valores	N	12503	16673	25013	33343	49999	44444
1	-	nColisionesI		9485	9485	9485	9485	9485	9485
1	-	nPasosExtraI		43756563	43756551	43756551	43756551	43756551	43756551
2	256	nColisionesI		3933	2973	2035	1453	**953**	1238
2	256	nPasosExtraI		18512	7896	3404	2106	**1206**	1657
3	500	nColisionesI		4044	3026	1963	1579	974	1304
3	500	nPasosExtraI		20045	7426	3173	2258	1221	1695
3	503	nColisionesI		3932	3002	2019	1480	**945**	1113
3	503	nPasosExtraI		18398	7677	3487	2140	**1174**	1437
3	1024	nColisionesI		4071	2988	2021	1499	972	1266
3	1024	nPasosExtraI		23221	7353	3426	2153	1205	1654
3	1031	nColisionesI		4024	2993	2068	1531	982	1170
3	1031	nPasosExtraI		21590	7650	3431	2168	1241	1586

Vemos una pequeña disminución en el número de colisiones y pasos extra necesarios cuando utilizamos la dirección de correo electrónico en lugar del alias del usuario. Pero esta variación (sobre un 5%) es poco significativa en el proceso en general, si la comparamos con la correcta elección de un buen valor de N o de una buena función Hash.

En particular para la función Hash 1, seguimos encontrando una gran cantidad de colisiones, que entendemos sigue estando relacionada con el método que se emplea en esta función (simple suma de los códigos ascii) y que todas las direcciones de correo tienen un formato similar (longitudes similares, algunas mayúsculas, casi todo minúsculas, puntos, y @vacunas.gal al final como nombre de dominio). En un entorno real con algo mayor de variabilidad en las direcciones de correo, es de suponer que los resultados hubieran sido algo mejores, pero tampoco demasiado.

3. *Influencia de estrategia de recolocación en el proceso de inserción*

En este nuevo apartado se nos pide que seleccionemos un par de casos de tamaño N y función hash que hayan funcionado bien en conjunto (produciendo pocas colisiones). Estos han sido los elegidos:

Caso 1: N=49999 (primo, L=0.2), Función Hash=2 (K=256)

Caso 2: N=49999 (primo, L=0.2), Función Hash=3 (K=503)

Ambas combinaciones han obtenido muy buenos resultados en la estrategia de recolocación.

Se nos solicita que, para los casos 1 y 2, estudiemos el comportamiento del número de colisiones en la inserción (nColisionesI) y el número de pasos extra (nPasosExtraI) al variar la estrategia de recolocación: simple (lineal con a=1), lineal (con al menos dos valores distintos de la constante a) y cuadrática. Como valores adicionales para a, ante la falta de alguna referencia, hemos elegido el 7 y el 15.

Adicionalmente a lo solicitado en la práctica, comprobaremos también el efecto del uso como clave del nickname o del e-mail. Los resultados obtenidos se pueden observar en la siguiente tabla:

TABLA 6
INFLUENCIA DE LA ESTRATEGIA DE RECOLOCACIÓN EN EL PROCESO DE INSERCIÓN

Clave	N	Func. Hash	K	Variables	Simple (lineal, a=1)	Lineal (a=7)	Lineal (a=15)	Cuadrática
ALIAS	49999	2	256	nColisionesI	1046	1047	1062	1040
ALIAS	49999	2	256	nPasosExtraI	1344	1319	1326	1290
ALIAS	49999	3	503	nColisionesI	**922**	**946**	**936**	**931**
ALIAS	49999	3	503	nPasosExtraI	**1168**	1210	**1196**	**1164**
CORREO	49999	2	256	nColisionesI	953	935	953	947
CORREO	49999	2	256	nPasosExtraI	1206	**1168**	**1178**	**1160**
CORREO	49999	3	503	nColisionesI	**945**	**945**	**940**	**942**
CORREO	49999	3	503	nPasosExtraI	**1174**	**1158**	**1173**	**1163**

Vemos que hay poca variación entre utilizar un método de recolocación u otro (simple, lineal o cuadrático). Sí que notamos que se obtienen mejores valores con K=503 en comparación del otro valor de K seleccionado.

En cuanto al uso de un campo u otro como clave, el correo electrónico ha tenido de nuevo datos bastante parecidos a los del alias.

4. *Estimación de la eficiencia en el acceso a los datos (búsqueda)*

Se nos pide finalmente que comparemos el número de pasos requeridos en la búsqueda de los elementos de la tabla. Vamos a buscar todos los elementos una vez insertados, y comprobar el número total de operaciones extra necesarias para encontrarlos (nOperacionesExtraB / nPasosExtraB).

Para ello, hemos aprovechado algunos de los cálculos previamente realizados, donde ha habíamos guardado para este apartado el valor del número de operaciones adicionales que se tienen que realizar para buscar todos los elementos una vez insertados. Hemos tenido además que calcular algunos casos adicionales que no habían sido aún procesados en el apartado 3.

En esta ocasión siempre hemos usado el nickname (alias) como clave para el hash. Los resultados obtenidos se muestran en esta tabla:

TABLA 7
INFLUENCIA DE N EN EL PROCESO DE BÚSQUEDA (NOPERACIONESEXTRAB)

Estrategia impl.	Func. Hash	K N	12503	16673	25013	33343	49999	44444
Encadenamiento	1	-	129176	129176	129176	129176	129176	129176
Encadenamiento	2	256	4033	2827	2032	1504	**1060**	4560
Encadenamiento	3	500	3989	2987	2009	1543	**1066**	4445
Encadenamiento	3	503	4078	2830	2049	1435	**944**	1112
Recol, simple (a=1)	1	-	46720557	46720557	46720557	46720557	46720557	46720557
Recol, simple (a=1)	2	256	21576	6824	3488	2029	1344	4606
Recol, simple (a=1)	3	500	21122	6729	3511	2136	1449	4466
Recol, simple (a=1)	3	503	20387	6787	3507	1967	**1168**	1436
Recol, lineal (a=7)	1	-	6677618	6677618	6677618	6677618	6677618	6677618
Recol, lineal (a=7)	2	256	21380	6437	3387	2062	1319	4610
Recol, lineal (a=7)	3	500	20167	7527	3342	2216	1319	4477
Recol, lineal (a=7)	3	503	20221	7171	3226	2055	1210	1396
Recol, lineal (a=15)	1		3122647	3122647	3122647	3122647	3122647	3122647
Recol, lineal (a=15)	2	256	22897	7451	3217	2145	1326	4600
Recol, lineal (a=15)	3	500	24885	7238	3410	2135	1546	4468
Recol, lineal (a=15)	3	503	19895	6681	3456	2072	**1196**	1423
Recol. cuadrática	1		635109	635029	635029	635029	635029	635029
Recol. cuadrática	2	256	11846	5550	3066	1981	1290	5478
Recol. cuadrática	3	500	11593	5822	3040	2091	1396	5270
Recol. cuadrática	3	503	11440	5654	3144	1912	**1164**	1407

En cuanto al proceso de búsqueda, vemos que de nuevo la función Hash 1 proporciona los peores resultados en el proceso de búsqueda. Entre el resto de los parámetros, que más influye a la hora de obtener un número de operaciones extra bajo vuelve a ser el valor de N (y el 49999, con un L= 0.2 vuelve a ser nuestro ganador).

Entre las funciones hash 2 y 3, sin importar el valor de K, no hay grandes diferencias, aunque K=503 ha dado resultados algo mejores.

En cuanto a la estrategia de implementación de encadenamiento, esta vez obtiene resultados algo mejores que la estrategia de recolocación. En cuanto a esta última, no ha habido diferencias importantes entre la de recolocación simple, la lineal con valores mayores de salto o la cuadrática.

V. CONCLUSIONES

Sen han analizado la repercusión de estas posibilidades:

- Dos implementaciones distintas la tabla hash (encadenamiento y recolocación)
- Tres diferentes funciones de Hash (suma directa vs suma ponderada con distintos valores de K)
- 6 tamaños distintos tamaños de la tabla (comparando algunos más "acertados" con otros menos "adecuados" para N).
- La selección de distintos campos como clave para el hash (nickname/alias vs e-mail).
- Distintas estrategias de recolocación (la simple, la lineal y la cuadrática).
- Se han calculado los valores obtenidos tanto durante el proceso de inserción en las tablas hash como en las posteriores búsquedas en dichas tablas.

Hemos visto que **la mayor repercusión** a la hora de seleccionar los distintos criterios está en **una buena función de Hash**. Descartaremos para este tipo de casos la función de suma "simple" (función hash 1) por generar una gran cantidad de colisiones, y **nos quedaremos con la de suma ponderada (funciones hash 2 y 3)**. En cuanto al factor de multiplicación, aunque hay pequeños cambios en favor de algún número más raro como el 503, vemos que no hay grandes diferencias.
El otro gran elemento diferenciador de cara a tener buenos resultados, tanto en la inserción como en la búsqueda, es la elección de un tamaño (N) adecuado para la tabla. Entre los que hemos seleccionado, **ha resultado ganador el número primo que tenía un menor factor de carga (el 49999 con L=0.2)**. Un factor de carga más alto o un número no primo afectan negativamente a los resultados.

Entre una tabla hash que funcione con un profeso de inserción en encadenamiento y otra que funcione con un proceso de inserción en recolocación, aunque la diferencia entre ambas es pequeña (si no modificamos el resto de los parámetros), **nos quedaríamos con la de inserción en encadenamiento, por haber obtenido mejores resultados en general**.

Por lo tanto, para los datos proporcionados la mejor estrategia de implementación, tanto de cara a la inserción como a la búsqueda, sería la de encadenamiento con un tamaño grande (valor de N, y por lo tanto factor de carga relativamente bajo), una función de hash basada en la suma ponderada.